# Google Image Search Engine

## A Project Report

Submitted in partial fulfilment of the

Requirements for the award of the Degree of

**MASTER OF SCIENCE**

**(INFORMATION - TECHNOLOGY)**

**By**

**6961 SHARON PHILIP**

**Under the esteemed**

**Guidance of**

**Prof. RASHMI SUBODH CHAVAN**

**Designation: Assistant Professor**

**DEPARTMENT OF COMPUTER SCIENCE MAHATMA EDUCATION SOCIETY'S**

**PILLAI COLLEGE OF ARTS, COMMERCE AND SCIENCE (Autonomous), NEW PANVEL, 410206, MAHARASHTRA**

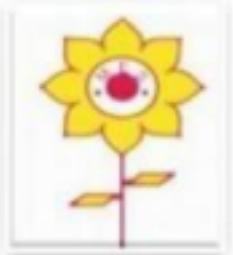
**(Affiliated to University of Mumbai) 2024-2025**
**MAHATMA EDUCATION SOCIETY'S**

*PILLAI COLLEGE OF ARTS, COMMERCE AND SCIENCE (Autonomous) NEW PANVEL, 410206, MAHARASHTRA*

**(Affiliated to University of Mumbai)**

**DEPARTMENT OF COMPUTER**

**SCIENCE**



## CERTIFICATE

This is to certify that the project **entitled "Google Image Search Engine"** is Bonafide work of **Sharon Philip** bearing Roll. No **6961**. submitted in fulfillment for the completion of MSc. degree in **Information Technology**, University of Mumbai.

**Internal Guide** **Co-Ordinator**

*Date:* *College seal* *External Examiner*

# ACKNOWLEDGEMENT

I Mr. **SHARON PHILIP** student of PILLAI COLLEGE OF ARTS, COMMERCE & SCIENCE (AUTONOMOUS), NEW PANVEL would like to express My sincere gratitude towards our college's Computer Science Department.

I would like to thank our Coordinator **Prof. RASHMI CHAVAN** for her constant support during this project and granting me the opportunity to build a project for the college. The project would have not been completed without the dedication, creativity and the enthusiasm my family provided me.

Yours faithfully,
**SHARON PHILIP**
(Final Year Information Technology)

# DECLARATION

I declare that this written submission represents my ideas in my own words and where others ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea / data / fact / source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

**Name and Signature of the Student**

**SHARON PHILIP**

## ABSTRACT

Image search engine is a specialized search engine for retrieving images. It is rapidly growing with the growth in digital images on the web. The essential role of image search engine is to retrieve image results that are relevant to the users and provide them more reliable and comfortable services. The main challenge for image search engine is to retrieve images with content matching much more than text matching. This paper investigates the performance of three image search engines. The paper is realized in two phases. In the first phase, three image search engines, namely, Google, Yahoo and Ask, are selected. Then, twenty queries are determined from two researches. Each query is run on each image search engine separately and first twenty images retrieved are classified as being relevant or non-relevant. Afterwards, precision ratios are calculated at various cut-off points. In the second phase, image features, namely, colour and shape are determined. Some of the images retrieved from first phase are analysed according to their features. The results of first phase indicated that Google has the best overall retrieval effectiveness with 95% precision ratio, followed by Yahoo with 91%, and Ask at the last with 83.7%. Furthermore, the results from second phase showed that two images with similar colour histograms can possess different content. As well as for shape feature, the number of edges is not efficient to identify relevant images.


Keywords: Search Engine (SE): Content-Based Image Retrieval (CBIR); Image Features.

# TABLE OF CONTENTS

# CHAPTER-1

## 1 INTRODUCTION

World Wide Web plays an essential role in human life. A large amount of information is available on the web. So search engines play an important role in allowing users search the contents online using textual queries Information retrieval systems have a critical role in fully utilizing information available on the internet. Most web search engines are built based on traditional information retrieval technologies, which essentially focus on text-based information retrieval. As digital technologies enhance and the spread of multimedia information, images become increasingly important for users to visualize things and concepts

With the rapid growth in digital images and increase of users searching images on the web, it's important to understand the image requested by users and design feasible solutions to deal with the large variety of needs and uses on a large scale [21]. Despite the popularity of digital images and it’s rapid

Increase on the internet, there is little attention that has been paid on image retrieval systems for web use this makes the task of image retrieval on the web more complicated. The type of search engine specialized on finding images, pictures and animations is called Image Search Engine or Image Search

The main objective of ISE is to retrieve image results that are relevant to the requested query and diverse enough to cover differences of semantic or visual concepts. The relevant images in traditional search engines found by matching the text query with image metadata (i.e. anchor text, surrounded text) Many top ranked images may be irrelevant to Open withiest information is limited and can be inaccurate. Also, there is no reliable way to actively promote the variation of the results without analysing content of the images so there is a need to improve image search results and make the retrieval more efficient and accurate.

The image content has essential role in retrieval process; there are many visual features that are used to describe content of images (colour, shape, and texture). The user queries need to be interpreted correctly in image search engine, so we need to develop an approach that helps the user to reach his search goals faster and more comfortably.

The objective of this paper is to present an approach that improves information retrieval in image search engines. This paper evaluates the performance and competence of several ISEs by calculating the recall and precision to measure the effectiveness of ISES related to images retrieval and compare them. For the images that are retrieved from ISEs, we need to analyse their features. So we used MATLAB program to analyse the common image features and evaluate the ISES according to these features. This will identify the

Searchable image's features and then analyse the Impact of these features on the retrieval.

## 1.1 BACKGROUND:

An **image search engine** is a technology that allows users to search and retrieve images from a database based on visual input or textual queries. The development of image search engines is rooted in the need to organize and make vast collections of images accessible in an efficient and meaningful way.

**Evolution of Image Search Technology**

1. **Text-Based Search (Early Days)**:

- Initially, image search engines relied solely on text-based approaches where users searched using keywords or tags associated with images.
- Images were indexed based on **metadata** (e.g., filenames, alt text, captions), meaning only images tagged with relevant text could be retrieved.

2. **Content-Based Image Retrieval (CBIR)**:

- The limitations of text-based methods led to the development of **Content-Based Image Retrieval (CBIR)** in the 1990s.

- CBIR introduced the ability to search based on the actual content of the images, such as color, texture, shape, and objects.

## 1.2 OBJECTIVE:

The purpose of this project is to make easier for people to get out their essential thing they want. Things or the information is arranged in that way people can easily interact with them. The main purpose is to giving quick response to their requirement and save their time.
A Image search engine is a software program that helps people find the information they are looking for online using keywords or phrases.

By achieving these objectives, an image search engine can provide users with fast, accurate, and scalable search functionality, whether for personal use (e.g., finding similar photos) or professional applications (e.g., e-commerce, digital libraries).

## 1.3 PURPOSE:

The purpose of this project is to make easier for people to get out their essential thing they want. Things or the information is arranged in that way people can easily interact with them. The main purpose is to giving quick response to their requirement and save their time.
A search engine is a software program that helps people find the information they are looking for online using keywords or phrases.

## 1.4 SCOPE:

In the modern world of the technology, the scope of this software is very high and many new changes and updates will provide in future. In current scenario people doesn't get their resulting output

or the thing they want on internet. So, because of this they may lose their precious time for such things.

## 1.5 **Applicability.**

The image search engine has diverse applicability across multiple industries, from retail and media to healthcare and law enforcement. Its ability to search, analyse, and retrieve images based on visual content opens up opportunities for innovation in product discovery, research, creativity, and security applications.

# Chapter 2 – PROPOSED SYSTEM

## *1. System Overview*

The proposed image search website allows users to search for images by entering keywords. Images are fetched from the Unsplash API, providing high-quality and royalty-free visuals.

## *2. Key Features*

- **Keyword Search**: Users can enter keywords to find relevant images from Unsplash.
- **Image Display**: Display a grid of fetched images with options to view more details.
- **Responsive Design**: A mobile-friendly interface for seamless use across devices.
- **Lightbox Feature**: Users can click on images to view them in a larger format.

## *3. System Architecture*

- **Frontend Layer**:
    - Built using HTML, CSS, and JavaScript frameworks (React, Angular, or Vue.js).
    - Provides the user interface for inputting keywords and displaying image results.
- **Backend Layer**:
    - Optionally, you can use a simple to manage API requests.
    - Directly interact with the Unsplash API for fetching images based on user input.
- **API Integration**:
    - Connect to the Unsplash API to retrieve images based on the user's keyword search.
    - Handle API requests and responses effectively to ensure fast and accurate results.

### *4. Proposed Workflow*

1. **User Interaction**:
    - Users enter a keyword in the search bar and submit the query.
2. **API Request**:
    - The frontend sends a request to the Unsplash API with the provided keyword.
3. **Image Fetching**:
    - The Unsplash API responds with a list of images related to the keyword.
    - Images are returned along with metadata (e.g., author, description).
4. **Results Presentation**:
    - The website displays the fetched images in a grid format.
    - Users can click on images to view them in a lightbox for a closer look.

### *5. Technical Requirements*

- **Frontend Development**:
    - HTML, CSS, JavaScript (or a framework like React).
    - Use Axios or Fetch API for making HTTP requests to Unsplash.
- **API Integration**:
    - Register for an Unsplash API key to access their image database.
    - Handle API rate limits and pagination for efficient image fetching.

### *6. Security Measures*

- Ensure API keys are stored securely (e.g., environment variables).
- Implement HTTPS to secure data transmission.

### *7. Future Enhancements*

- **User Accounts**: Allow users to save favorite images or create collections.
- **Advanced Filters**: Enable filtering by image orientation, color, or category.
- **Image Download**: Provide options for users to download images directly.

This image search website utilizing the Unsplash API aims to deliver a simple yet effective platform for users to search for and explore high-quality images. By focusing on user experience and efficient API integration, the site can meet the needs of individuals and businesses looking for compelling visuals.

# Chapter 3 – IMPLEMENTATION METHODS

This implementation method outlines the steps necessary to build an image search website that fetches images based on user-input keywords from the Unsplash API. Below is a detailed breakdown of each component involved in the development process.

## *1. Project Setup*

**Development Environment**:

- **Code Editor**: Choose a code editor like Visual Studio Code, which provides a rich set of features for web development.
- **Local Development Server**: You can use Node.js with Express for a dynamic backend or a simple static server (like Live Server in VS Code) for this project, since the primary focus is on front-end interaction with the Unsplash API.

## *2. Frontend Development*

**HTML Structure**:

- **Basic Layout**: The index.html file contains the main structure of the web page:
  - A header with an input field for keywords and a button to trigger the search.
  - A main section where the search results will be displayed.
- **Semantic Elements**: Using <header>, <main>, and <h1> tags improves accessibility and SEO.

**CSS Styling**:

- **Visual Appeal**: The styles.css file styles the layout to create a user-friendly experience:
  - A grid layout for displaying images, ensuring they are responsive and visually appealing.
  - Basic styles for body, images, and the results container.

**JavaScript Functionality**:

- **Dynamic Interactivity**: The script.js file manages user interactions and handles API requests:
  - **Event Listener**: When the user clicks the search button, the fetchImages function is triggered.

- **API Request**: The fetch Images function constructs a URL with the user's keyword and makes an asynchronous request to the Unsplash API using the Fetch API.
- **Data Handling**: Once the response is received, the function processes the JSON data and passes the image results to the display Images function.

### *3. API Integration*

**Unsplash API**:

- **API Key**: Sign up at Unsplash and obtain an API key, which is necessary for making requests.
- **Fetching Images**: The API endpoint for searching photos is constructed with the user's query and the API key. The results include relevant images and their metadata.

**Displaying Images**:

- The display Images function updates the DOM by creating image elements and appending them to the results container. This allows the user to see the fetched images in real time.

### *4. Testing the Application*

- **Local Testing**: Run the application in your browser to ensure all components work correctly:
  - Test various keyword inputs to confirm that images are fetched and displayed appropriately.
  - Check for errors in the console to troubleshoot any issues that arise during the API requests.

### *5. Deployment*

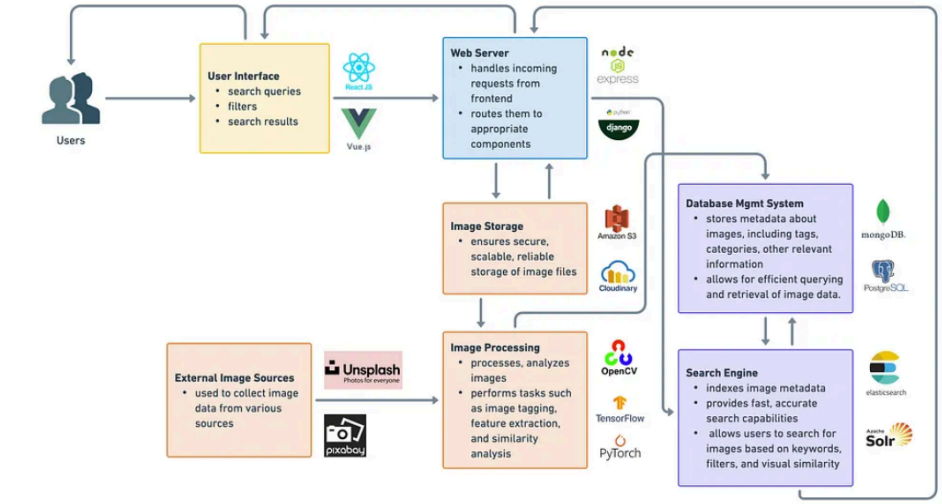
- **Web Hosting**: Choose a hosting platform to deploy your website:
  - **GitHub Pages**: Good for static sites; push your code to a repository and enable GitHub Pages in the settings.
  - **Netlify/Vercel**: These platforms provide easy deployment options for static sites and integrate well with GitHub.
- **API Key Security**: Ensure that your API key is stored securely and not hard-coded into the public codebase. You can use environment variables or server-side logic to manage sensitive information.

### *6. Future Enhancements*

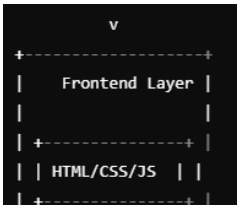
- **Lightbox Feature**: Implement a lightbox for viewing images in full size when clicked, enhancing user experience.
- **Pagination**: Add pagination controls to manage large sets of search results and improve load times.
- **Error Handling**: Implement robust error handling for various scenarios, such as no results found or network issues, to provide better user feedback.
- **Advanced Filters**: Allow users to filter results by parameters such as image orientation (landscape, portrait) or dominant color for more refined searches.

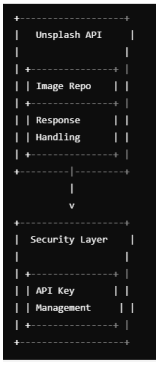
This implementation method provides a comprehensive approach to building an image search website using the Unsplash API. By following these steps, you can create a functional, user-friendly platform that allows users to search and discover high-quality images effortlessly. The structure is designed to be scalable and adaptable, allowing for future enhancements and features to improve user engagement and experience.

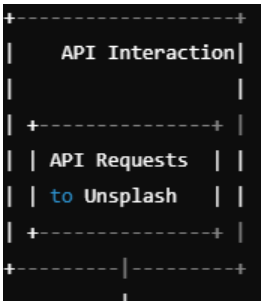
## SYSTEM DESIGN




User Interface
Web Browser
Search Bar
Image Display


Frontend Layer
HTML/CSS/JS


Unsplash API
Image Repo
Response Handling
Security Layer
API Key Management


API Interaction
API Requests to Unsplash

### User Interface (UI)

- **Web Browser**: Users interact with the application via a web browser.
- **Search Bar**: Allows users to enter keywords for image searches.
- **Image Display Area**: Shows the fetched images in a grid layout.

### Frontend Layer

- **HTML/CSS/JavaScript**: Technologies for building the user interface.
- **Framework (Optional)**: Use of frameworks like React or Angular for a dynamic and responsive user experience.

### API Interaction

- **API Requests**: Frontend sends requests to the Unsplash API based on user input.

### Backend Layer (Optional)

- **Web Server**: Node.js/Express or Flask to handle requests and responses.
- **API Gateway**: (If implemented) to manage communication between the frontend and Unsplash API, potentially allowing for caching and rate limiting.
- **Database (Optional)**: To store user data and favorites if user accounts are implemented.

### Unsplash API

- **Image Repository**: Provides access to high-quality images based on search queries.
- **Response Handling**: Returns image data (URLs, metadata) to the frontend.

### Security Layer

- **API Key Management**: Secure handling of the Unsplash API key, ensuring it is not exposed in the frontend.

# Algorithm

The image search algorithm in your Unsplash-based search engine would depend on how you process user input, send it to the Unsplash API, and present the returned images. The core search functionality, including ranking and relevance, is managed by Unsplash itself, but you can enhance the search experience with additional logic on the client-side.

Here's a high-level algorithmic breakdown for your image search engine:

**1. Input Parsing:**

- **User Input**: The user enters a search query (keyword or phrase) in the search bar.
- **Keyword Normalization**: Ensure proper normalization of the input by trimming spaces and converting the text to lowercase for consistent API requests.

**2. API Request to Unsplash:**

- **Construct the Request**: Use the normalized query to construct an API request to Unsplash. Use parameters like the query keyword, number of results per page, and other filtering options (if available).
- **Pagination Handling**: If the number of results exceeds the maximum per page (usually 30 by default), handle pagination by requesting the next page upon user scroll or interaction.

**3. Processing the API Response:**

- **Retrieve Results**: The Unsplash API will return a list of images matching the search query.
- **Data Extraction**: Extract useful data from the response, including:
  - Image URLs (different sizes)
  - Photographer name
  - Alt description
  - Image dimensions
  - Image metadata (if needed, like downloads count or likes)

**4. Image Display:**

- **Grid Layout**: Display the images in a visually appealing grid format. Each image should show a thumbnail, with the option to view more details or download the image.

- **Lazy Loading**: Use lazy loading to improve performance when there are many images by loading more images as the user scrolls down.

## 5. Post-Processing for Better Experience:

- **User Session Management**: Save users' search history, so they can revisit previous queries.
- **Image Previews**: Offer hover previews or lightbox views for larger image previews without loading a new page.

By following this approach, the image search engine integrates smoothly with the Unsplash API while enhancing user experience through filters, pagination, and optimized display techniques.

# CHAPTER 4 – DEVELOPMENT TOOLS

An **Image Search Engine using the Unsplash API** built with **HTML5, CSS3, and JavaScript** refers to a web application that allows users to search for images by typing keywords, then fetches high-quality and royalty-free images from the Unsplash database using their API. Here's how each language plays a role:

## 1. Setup the Project in VS Code

### *1.1. Install Visual Studio Code*

If you haven't already, download and install **Visual Studio Code** from here.

### *1.2. Create a New Project Folder*

1. Open **VS Code**.
2. Create a folder for your project called image-search-engine.
    - In **VS Code**, go to **File > Open Folder** and select the image-search-engine folder.

### *1.3. Create Project Files*

Inside the image-search-engine folder, create three files:

- index.html (HTML structure)
- style.css (CSS styles)
- app.js (JavaScript logic)

Your project structure should look like this:

arduino

```
/image-search-engine
├── index.html
├── style.css
└── app.js
```

### 2. HTML: Build the Structure

In **VS Code**, open the index.html file and add the following HTML code:

## Code

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Image Search Engine</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <h1>Image Search Engine</h1>
    <form id="search-form">
        <input type="text" id="search-box" placeholder="Search Here">
        <button>Search</button>
    </form>
    <div id="search-result"></div>
    <button id="show-more-btn">Show more</button>

    <script src="script.js"></script>
</body>
</html>
```

- **Explanation**:
    - The **search container** contains an input field for the search query and a button to trigger the search.
    - The **image grid** is where the search results (images) will be displayed.
    - The **Load More** button allows users to load more images if needed.

## CSS: Style the Page

In **VS Code**, open the style.css file and add the following CSS code:

### Code

```
*{
    margin: 0;
    padding: 0;
    font-family: 'Poppins', sans-serif;
    box-sizing: border-box;
}

body{
    background: rgb(63, 6, 63);
    color: white;
}

h1{
    text-align: center;
    margin: 50px auto 30px;

}

form{
    width: 90%;
    max-width: 600px;
    margin: auto;
    height: 60px;
    background: rgb(88, 34, 91);
    display: flex;
    align-items: center;
    border-radius: 20px;
}

form input{
    flex: 1;
    height: 100%;
    border: 0;
    outline: 0;
    background: transparent;
    color: #fff;
    font-size: 18px;
    padding: 0 30px;
}

form button{
    padding: 0 40px;
    height: 100%;
```

```
    background: #c20cc8;
    color: #fff;
    border: 0;
    outline: 0;
    border-top-right-radius: 20px;
    border-bottom-right-radius: 20px;
    cursor: pointer;
}

::placeholder{
    color: #fff;
    font-size: 18px
}

#show-more-btn{
    background: #c20cc8;
    color: #fff ;
    border: 0;
    outline: 0;
    padding: 10px 20px;
    border-radius: 4px;
    margin: 10px auto 100px;
    cursor: pointer;
    display: none;
}

#search-result{
    width: 90%;
    margin: 70px auto 50px;
    display: grid;
    grid-template-columns: 1fr 1fr 1fr;
    grid-gap: 40px;
}

#search-result img{
    width: 100%;
    height: 230px;
    object-fit: cover;
    border-radius: 5px;
}
```

**Explanation**:

- The **image grid** layout is styled using CSS Grid to make it responsive.
- Buttons have hover effects, and the image layout includes zoom effects when hovering over an image.
- The **Load More** button is initially hidden and only displayed when there are more images to load.

### JavaScript: Implement API Logic

In **VS Code**, open the app.js file and add the following JavaScript code:

## Code

```
const accessKey = "vNqsZ6dKyeSQn5dpF5U69njlUoxB3727W1NpPEiYlpc";

const searchForm = document.getElementById("search-form");
const searchBox = document.getElementById("search-box");
const searchResult = document.getElementById("search-result");
const showMoreBtn = document.getElementById("show-more-btn");

let keyword = "";
let page = 1;

async function searchImages() {
    keyword = searchBox.value;
    const url =
`https://api.unsplash.com/search/photos?page=${page}&query=${keyword}&client_i
d=${accessKey}&per_page=12`;

    const response = await fetch(url);
    const data = await response.json();

    if(page === 1) {
        searchResult.innerHTML = "";
    }

    const results = data.results;

    results.map((result) =>{
        const image = document.createElement("img");
        image.src = result.urls.small;
        const imageLink = document.createElement("a");
        imageLink.href = result.links.html;
        imageLink.target = "_blank";

        imageLink.appendChild(image);
        searchResult.appendChild(imageLink);
    })
    showMoreBtn.style.display = "block";
}

searchForm.addEventListener("submit", (e) => {
    e.preventDefault();
    page = 1;
    searchImages();
})
```

```
showMoreBtn.addEventListener("click",()=>{
    page++;
    searchImages();
})
```

- **Explanation**:
    - This script connects to the **Unsplash API** using your **Access Key**. Replace 'YOUR_UNSPLASH_ACCESS_KEY' with your actual key (which you can obtain from Unsplash Developers).
    - The search query is sent to the Unsplash API, and the resulting images are dynamically added to the page.
    - The **Load More** button will appear if there are additional pages of results, allowing users to load more images.

**Run the Project in VS Code**

### *5.1. Use Live Server Extension*

- Install the **Live Server** extension in VS Code (search for **Live Server** in the extensions panel).
- Once installed, right-click on the index.html file in VS Code and select **Open with Live Server**.
- This will launch your project in the browser, and you can interact with your image search engine.

### *5.2. Testing the Image Search*

- Type a keyword (like "nature" or "city") in the search bar, and click the search button.
- The images from Unsplash will appear in the grid below.
- If there are more than 9 results, the **Load More** button will appear, allowing users to load additional images.

**Enhancements**

Here are some features you could add to enhance the project:

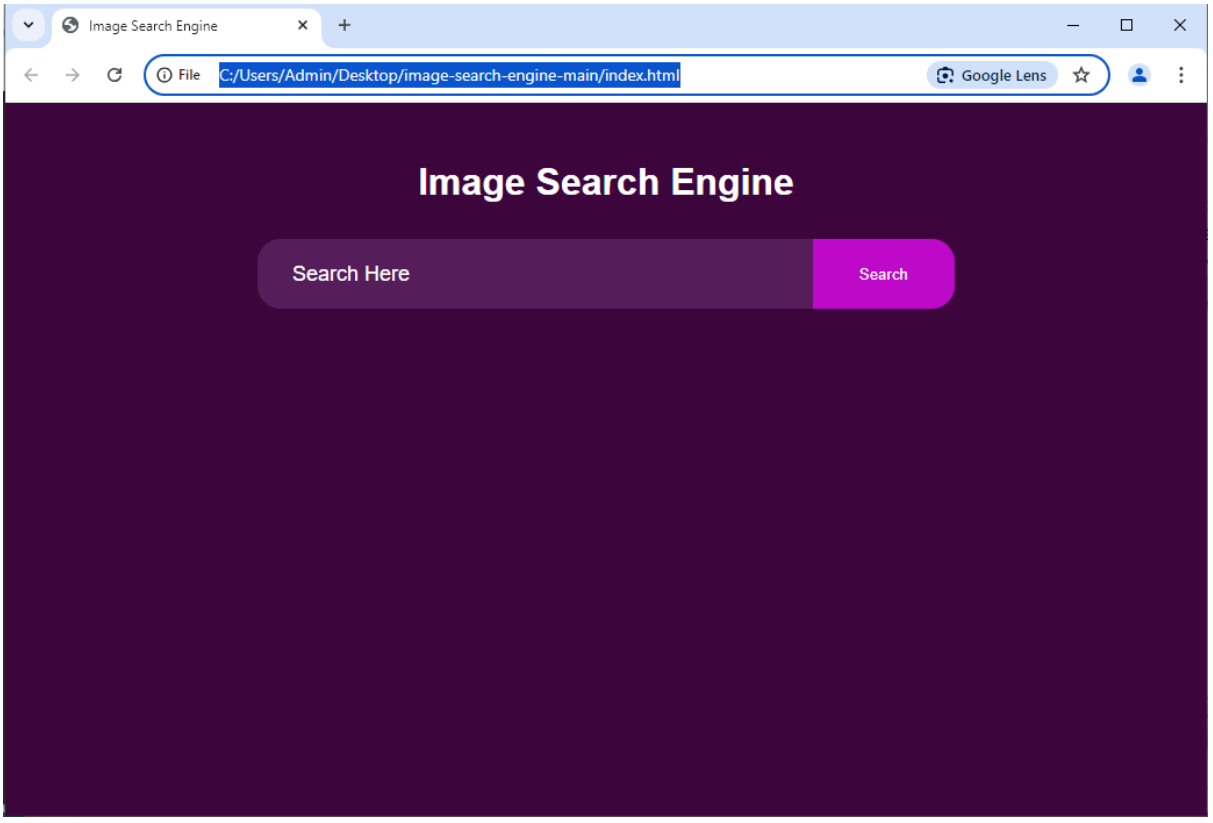
1. **Error Handling**: Display an error message if no images are found or the API call fails.
2. **Image Details**: Show photographer credits or the number of likes for each image.
3. **Responsive Design**: Further improve the responsiveness for mobile and tablet layouts.
4. **Search History**: Store and display past search queries for users to easily revisit.

By following these steps, you will have a functional image search engine using **Unsplash API** built with **HTML5, CSS3, and JavaScript** and run directly in **VS Code**.

# CHAPTER 5 – RESULTS AND DISCUSSIONS

Results and Discussions

## SEARCH PAGE



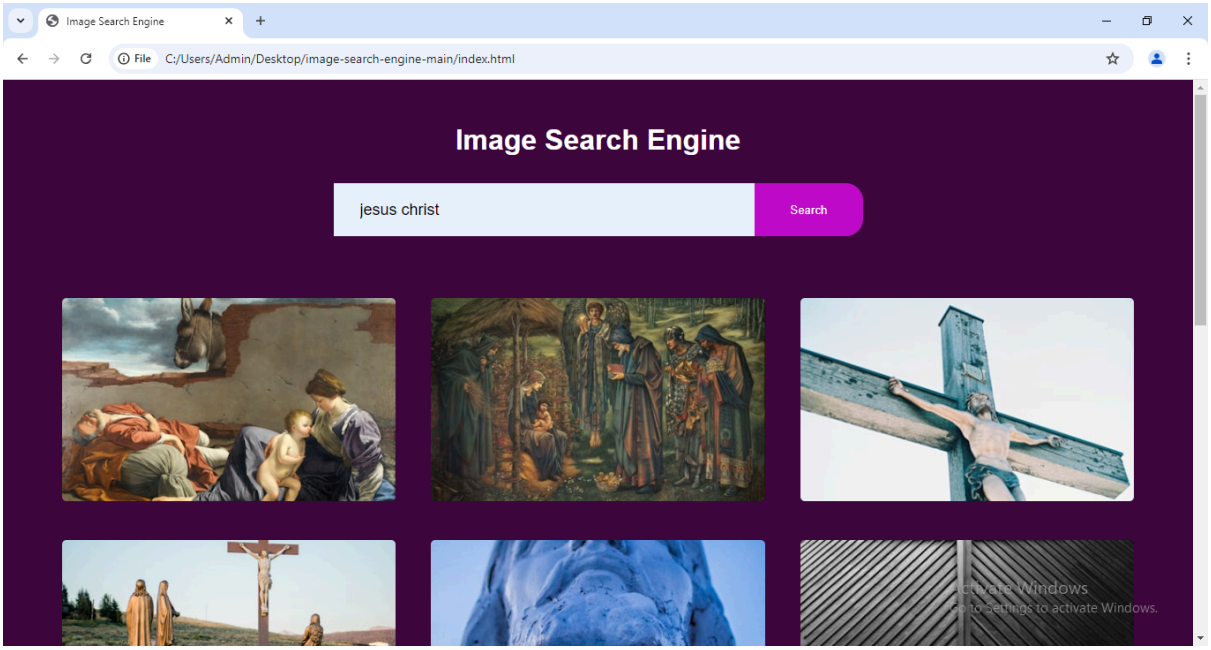


Image Search Engine
File C:/Users/Admin/Desktop/image-search-engine-main/index.html
Image Search Engine
jesus christ
Search
Activate Windows
Go to Settings to activate Windows.

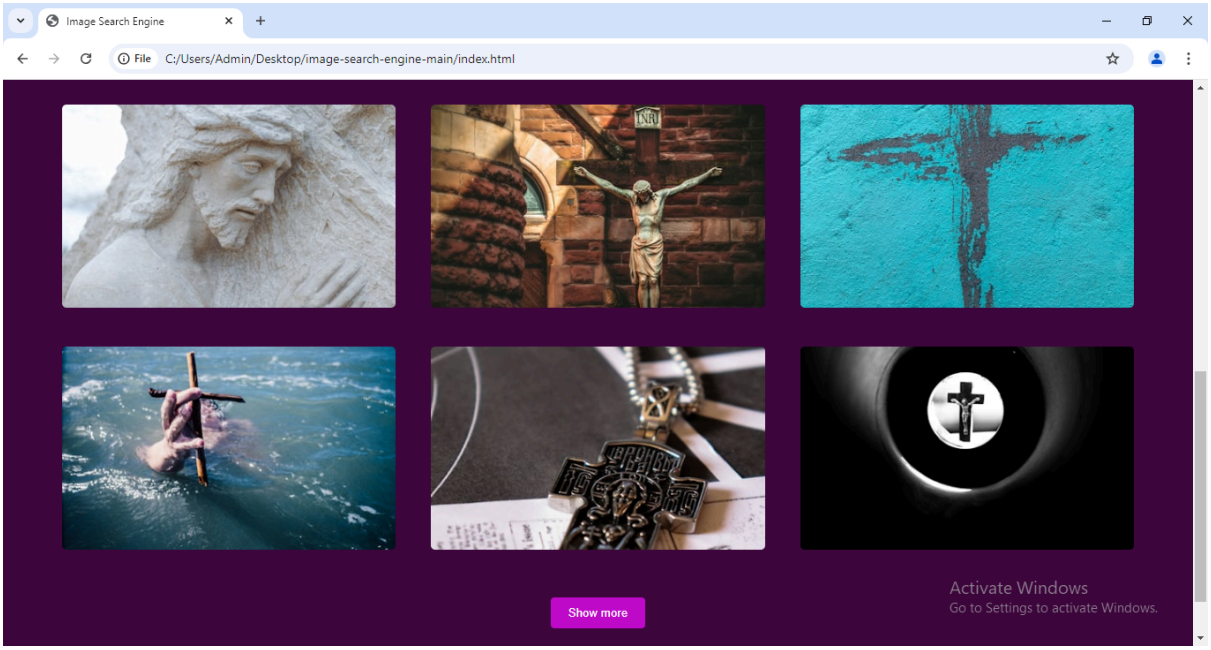


Image Search Engine
File C:/Users/Admin/Desktop/image-search-engine-main/index.html
Show more
Activate Windows
Go to Settings to activate Windows.

# CHAPTER 6 – FUTURE SCOPE

In the realm of an image search engine, a similar forward-looking vision as that of Snap-Tracks can be envisaged for its future progression and refinement. Enhancing the platform's image recognition capabilities through advanced algorithms, machine learning, and artificial intelligence is paramount. These technologies can refine the accuracy and relevance of search results, potentially allowing the search engine to interpret and understand diverse visual elements, objects, scenes, and even user context to offer more intuitive and precise results.

Integration of biometric authentication methods—such as facial recognition—could be explored for specific use cases like personal photo libraries or professional databases, ensuring enhanced security and personalized search experiences. The introduction of mobile applications would significantly elevate accessibility, offering users real-time search, offline capabilities, and push notifications for content updates, making image searches faster and more responsive on-the-go.

Data analytics and visualization promise to offer deeper insights into user behaviour and search trends. This could empower both users and businesses by providing customized image recommendations, optimizing marketing strategies, or aiding in content creation decisions. Additionally, the analysis of search patterns can improve image creation and foster deeper engagement by surfacing popular or trending images.

# CHAPTER 7 – CONCLUSION

There are millions of images on the web. ISEs help users in searching and retrieving images in a comfortable and easy way. This paper aimed to measure the effectiveness of three ISEs, namely, Google, Yahoo, and Ask. In addition, the images retrieved from these ISEs are analysed based on colour and shape features to determine the effect of these features on the retrieval.

The main findings of this paper can be summarized as follows: it seems that the precision ratio of any one of the ISEs was different and changed from one query to another. The precision ratios for the three ISEs decreased gradually with increasing cut-off point values from 10 to 20.

The results indicated also that Google has the best overall retrieval effectiveness with 95% precision ratio, Followed by Yahoo with 91% Precision ratio And ask at the last with 83.7% precision ratio.

Our results were compared with results from two previous studies [23, 2]. The comparison results showed that the queries (1, 3, 4, 6, 7, 8, 10 and 11) have the same results in two studies. However, the comparison showed that the results were different for queries (2, 5, 9 and 12 to 20) between our results and those in [23, 2].

Also, in this paper, the image features are analyzed and used to compare images to identify the impact of these features on the retrieval. Two features are selected, which are color and shape. However, there is one thing that's common for all ISEs which is the color feature is not sufficient to retrieve images that are relevant 100%, and two images with similar color histograms can possess different content. As well as for shape feature, the number of edges is not efficient to identify relevant images

# CHAPTER 8 – REFERENCES

**1.** Cakir, E., Bahceci, H., & Bitirim, Y., "An Evaluation of Major Image Search Engines on Various Query Topics," The Third International Conference on Digital Telecommunications, Bucharest, pp. 161-165, 2008.

**2.** Ben-Haim, N., Babenko, B., & Belongie, S., "Improving Web-based Image Search via Content Based Clustering," Conference on Computer Vision and Pattern Recognition Workshop, New York, USA, pp. 1-6, 2006

**3.** Fukumoto, T., "An Analysis of Image Retrieval Behavior for Metadata Type and Google Image Database," Information Processing and Management, vol. 42, no. 3, pp. 723-728, 2006. Gonzalez, R., & Woods, R., digital Image Processing, Addison-Wesley Longman, Boston, MA, USA, 2001.

**4.** Goodrum, A., "Image Information Retrieval: An Overview of Current Research," Journal of Informing Science, vol. 3, no. 2, pp. 63-67, 2000. Hassan, I., & Zhang. j., "Image Search Engine Feature Analysis," Online Information Review, vol. 25, no. 2, pp. 103-114, 2001.

**5.** Hiremath, P. S., & Pujari, J., "Content Based Image Retrieval Based on Color, Texture and Shape Features using Image and its Complement," International Journal of Computer Science and Security, vol. 1, no. 4, pp. 25-35, 2007.

**6.** Idrissi, K., Ricard, J., & Baskurt, A., "An Objective Performance Evaluation Tool for Color Based Image Retrieval Systems", Proceeding. 2002 International Conference on image processing, Rochester, New York, USA, pp. 389- 392, 2002.

**7.** qbal, Q., & Aggarwal. J. K., "Combining Structure, Color, and Texture for Image Retrieval: A Performance Evaluation," 16th International Conference on Pattern Recognition, Quebec, Canada, pp. 438-443, 2002.

**8.** Jeong. S., "Histogram-based Color Image Retrieval," Psych221/EE362 Project Report, pp. 1-21, 2001.

**9.** Jing, Y., & Baluja, S., "Page Rank for Product Image Search, International World Wide Web Conference Committee, New York, USA pp. 307-315, 2008.

**10.** Kaur, M., Bhatia, N., & Singh, S., "Web Search Engines Evaluation Based on Features and End- user Experience." International Journal of Enterprise Computing and Business Systems, vol. 1, no. 2, pp. 1-19, 2011.

**11.** Kekre, H. B et al., "Image Retrieval with Shape Features Extracted Using Gradient Operators and Slope Magnitude Technique with BTC," International Journal of Computer Applications, vol. 6, no. 8, pp. 28-33, 2010.

**12.** Lei, Z., Fuzong, L., & Bo, Z., "A CBIR Method Based on Color-spatial Feature," TENCON 99. Proceeding of the IEEE Region 10 Conference, Cheju Island, pp. 166-169, 1999.

**13.** Liu, H et al., "Effective Browsing of Web Image Search Results," in Proceedings of 6th ACM SIGMM International Workshop on Multimedia Information Retrieval, New York, USA, pp. 84- 90, 2004.

# THANK YOU